Encrypted Cloud Storage with Searchable Symmetric Encryption

Shravan Adash M (23BEC1149)
Preetish RK (23BEC1470)
Prajinesh P (23BEC1113)
Vellore Institute of Technology, Chennai

Abstract

The increasing adoption of cloud storage has created significant challenges in protecting the confidentiality of sensitive data while maintaining efficient data retrieval. Conventional cloud storage systems can encrypt files before outsourcing them to cloud servers, but encryption prevents traditional keyword-based searching and requires users to download and decrypt data before performing searches. Searchable Symmetric Encryption (SSE) provides a solution to this limitation by enabling keyword searches over encrypted data without requiring the cloud server to decrypt the stored files.

To address this limitation, this project proposes an Encrypted Cloud Storage with Searchable Symmetric Encryption framework that combines secure file encryption with privacy-preserving keyword search. The proposed framework employs symmetric encryption for protecting uploaded files and constructs a searchable encrypted index containing protected keyword representations. When a user performs a search, a cryptographic search token is generated from the requested keyword and secret key. The cloud server uses this token to search the encrypted index and return the corresponding encrypted files without obtaining the plaintext documents.

The proposed system will implement a simplified SSE architecture using Python and an interactive software interface. The effectiveness of the system will be evaluated using search latency, encryption and decryption time, index construction time, storage overhead, and search scalability across different dataset sizes. The project will also consider the information leakage associated with searchable encryption, including search and access patterns. Recent research in 2025 and 2026 has focused on dynamic updates, permission control, multi-keyword queries, forward and backward privacy, efficient deletion, and scalable searchable encryption for cloud environments. The proposed project aims to demonstrate how cryptographic search mechanisms can provide a balance between cloud-data confidentiality and practical data retrieval.

Keywords: Searchable Symmetric Encryption; Cloud Storage; Encrypted Search; Symmetric Encryption; Cryptographic Search Token; Secure Cloud Storage

I. Problem Statement Definition

Conventional cloud storage systems generally require sensitive files to be encrypted before they are outsourced to cloud servers. Although encryption provides confidentiality against unauthorized access, it also makes conventional keyword searching difficult because the server cannot directly inspect encrypted documents. As a result, users may need to download and decrypt large amounts of data before performing a search, resulting in additional communication and computational overhead.

Searchable Symmetric Encryption provides the capability to search encrypted data while maintaining confidentiality of the stored files. However, practical SSE systems must address several challenges,

including secure index construction, efficient search-token generation, search efficiency, dynamic data updates, and information leakage through search and access patterns.

Therefore, there is a need for an encrypted cloud-storage framework that can provide secure file storage while allowing authorized users to perform keyword searches without requiring the cloud server to decrypt the stored data. The proposed project focuses on developing a practical SSE-based cloud-storage system that balances data confidentiality, search functionality, and computational efficiency.

II. Introduction

The rapid growth of cloud computing and remote data storage has made cloud-based data outsourcing an important component of modern information systems. Organizations and individuals increasingly store documents and sensitive information on cloud servers because of their scalability, accessibility, and reduced infrastructure requirements. However, outsourcing data to third-party cloud servers introduces privacy and confidentiality concerns because users no longer have complete control over the infrastructure storing their information.

Conventional cryptographic mechanisms such as Advanced Encryption Standard (AES) can be used to encrypt data before uploading it to the cloud. Encryption protects the confidentiality of the stored information, but it also introduces a significant usability problem. Since the cloud server cannot access plaintext content, conventional keyword searches cannot be directly performed over encrypted files. Users must therefore retrieve and decrypt the data before searching, which becomes inefficient as the amount of stored information increases.

Searchable Symmetric Encryption (SSE) has emerged as a practical cryptographic approach for addressing this conflict between confidentiality and searchability. SSE allows a data owner to encrypt files and construct a searchable encrypted index. Instead of sending a plaintext keyword to the cloud server, the authorized user generates a cryptographic search token, which the server uses to locate matching encrypted documents.

A typical SSE system therefore consists of encrypted files, a searchable encrypted index, a secret key, and a search-token generation mechanism. The data owner encrypts the documents and creates the searchable index before outsourcing them. During a search operation, the authorized user generates a token corresponding to the required keyword. The cloud server processes the token against the encrypted index and returns the identifiers of matching encrypted documents.

Recent research has focused on improving the practical security and efficiency of SSE. In 2025, researchers proposed dynamic SSE systems with lightweight permission control, multi-user authorization, verifiable search, and stronger forward and backward privacy. In 2026, research has further investigated efficient conjunctive queries, non-interactive deletion, Bloom-filter-based indexing, and decentralized multi-client Boolean keyword search.

To address these challenges, this work proposes an Encrypted Cloud Storage with Searchable Symmetric Encryption framework. The proposed framework combines encrypted file storage with a searchable encrypted index and cryptographic search tokens. The effectiveness of the proposed system will be

evaluated using search performance, encryption and decryption latency, index construction overhead, storage requirements, and scalability across different numbers of encrypted documents.

III. Related Work

Hu et al. [1] proposed SEAC, a dynamic searchable symmetric encryption scheme with lightweight update-search permission control for cloud-assisted electronic protected health information sharing. The work addresses privacy concerns associated with cloud-based health information retrieval and allows authorized clients to perform search and update operations on encrypted data. The scheme introduces permission information together with secure indexes to control which clients can perform specific operations. The study demonstrates that practical SSE systems require not only encrypted search but also secure permission management for multiple users. However, the proposed framework focuses on a more complex multi-client environment, whereas the proposed project initially concentrates on a simplified encrypted cloud-storage model with searchable keyword retrieval.

Xu et al. [2] proposed a forward-secure multi-user and verifiable dynamic searchable encryption scheme for cloud storage within a zero-trust environment. The framework considers a potentially malicious cloud server and aims to provide privacy, forward security, and verification of search results. The approach uses cryptographic mechanisms including oblivious pseudorandom functions and hash-based state structures to protect search operations and encrypted-data updates. The work demonstrates the importance of protecting not only encrypted files but also the search process and query information. However, the complexity of such multi-user and verifiable mechanisms makes them more suitable for advanced implementations, motivating a simplified SSE architecture for the proposed project.

Yang et al. [3] investigated dynamic searchable symmetric encryption with efficient and complete access control for multi-user cloud computing. The study addresses the limitation that many existing dynamic SSE schemes cannot simultaneously enforce access control for both keyword retrieval and encrypted-data updates. The proposed approach demonstrates the importance of combining searchable encryption with authorization mechanisms in multi-user cloud environments. The proposed project adopts the basic SSE search mechanism while identifying access control as a potential extension for future development.

A 2025 study proposed a forward and backward private conjunctive dynamic searchable symmetric encryption scheme with reduced communication overhead. The work focuses on conjunctive keyword searches, where multiple keywords can be searched simultaneously, while reducing the amount of information revealed through updates and searches. The proposed construction uses a three-dimensional chain structure and demonstrates significant reductions in communication cost during multi-keyword searches. This work highlights the importance of reducing leakage and communication overhead in practical SSE systems.

Ren et al. [5] proposed a dynamic searchable symmetric encryption scheme supporting efficient conjunctive queries and non-interactive real deletion. The approach addresses the limitations of existing dynamic SSE schemes in handling multiple keywords and securely removing deleted ciphertexts. A cuckoo filter is used to improve conjunctive-query efficiency, while separate search databases are used to support real deletion. The study demonstrates that practical searchable encryption must consider not only search functionality but also secure data updates and deletion.

Zhang et al. [6] proposed a Bloom-filter-based dynamic symmetric searchable encryption scheme for cloud data. The work focuses on improving index construction and search efficiency while supporting dynamic updates. The proposed approach combines a reverse index based on B+ trees with Bloom filters to improve the efficiency and flexibility of encrypted-data searching. This demonstrates the importance of efficient encrypted-index structures when searchable encryption is applied to larger cloud datasets.

Wang et al. [7] proposed a decentralized multi-client Boolean keyword search system for encrypted cloud storage. The framework addresses the key-escrow problem in multi-client SSE and supports Boolean keyword queries while achieving sub-linear search complexity. Experimental evaluation was performed using the Enron dataset and demonstrated the practical feasibility of the proposed approach. The work represents a more advanced direction of SSE involving decentralized key management and multiple authorized clients.

Wu et al. [8] proposed MMKFB, a multi-client and multi-keyword searchable symmetric encryption scheme providing both forward and backward privacy. The scheme supports multi-keyword threshold queries in a multi-client setting using additive homomorphism, and is reported to be particularly efficient when a large number of files are updated at once. The work demonstrates that supporting multiple keywords and multiple clients simultaneously requires additional cryptographic machinery beyond a basic single-keyword construction. The proposed project implements conjunctive multi-keyword search by intersecting independent single-keyword result sets on the trusted client, which is simpler but does not provide forward or backward privacy.

Wu et al. [9] proposed an efficient verifiable searchable encryption scheme that conceals both the search pattern and the access pattern. The work addresses the limitation that existing pattern-privacy schemes incur substantial computational overhead, using random values to obfuscate polynomial evaluations and thereby avoiding expensive ciphertext multiplication operations, while additionally allowing the user to verify that returned results are complete and correct. This work is directly relevant to the present project, because search-pattern and access-pattern leakage are the principal forms of leakage retained by the implemented scheme; concealing them is identified as future work rather than a property claimed here.

Khan et al. [10] proposed BloomSec, a scalable and privacy-preserving searchable encryption scheme that applies Bloom filters to symmetric searchable encryption. The scheme addresses the communication overhead that arises when the server must transmit large lookup structures to the user, and stores an individual Bloom filter for each file rather than a single consolidated keyword-indexed structure, which improves performance for large datasets. The study demonstrates that the representation chosen for the encrypted index directly affects both search efficiency and communication cost, which is consistent with the storage overhead measured for the encrypted index in the present implementation.

Xue et al. [11] proposed an incentive-assisted multi-keyword ranked searchable encryption scheme with hidden attributes for community data sharing. The work combines multi-keyword ranked search with fine-grained access control and verifiable search results in a distributed peer-to-peer setting, and additionally introduces a points-based incentive mechanism to sustain participation. The scheme illustrates a direction in which searchable encryption is extended with result ranking, access control and participation incentives, all of which lie outside the single-owner scope of the proposed project.

Mohammed et al. [12] presented a review of searchable symmetric encryption functionalities, challenges and solutions, surveying approximately eighty schemes covering single-keyword, multi-keyword, fuzzy, ranked, semantic and dynamic search. The review organises these works chronologically and identifies the privacy gaps that remain in current encrypted-search implementations. It provides a useful classification of the design space and confirms that single-keyword static SSE, as implemented in the proposed project, remains the standard baseline from which the more advanced functionalities are derived.

IV. Proposed Methodology

Conventional cloud storage systems protect outsourced information through encryption, but encryption prevents direct keyword searching over stored files. Therefore, the proposed system aims to integrate secure file encryption with a searchable encrypted index so that authorized users can retrieve relevant encrypted documents without requiring the cloud server to decrypt them.

The proposed framework will contain a client-side encryption and search-token generation component and a simulated cloud-server component. The client will encrypt files before uploading them and generate an encrypted index based on the keywords contained within the documents. During a search operation, the client will generate a cryptographic search token corresponding to the requested keyword. The cloud server will use this token to search the protected index and return the matching encrypted file identifiers.

The proposed methodology consists of the following stages:

1. Key Generation: A secret symmetric key will be generated and maintained by the authorized client.

2. File Encryption: Uploaded documents will be encrypted using a symmetric authenticated encryption mechanism such as AES-GCM before being stored on the cloud server.

3. Keyword Extraction and Index Construction: Keywords will be extracted from the documents and associated with the corresponding file identifiers. A protected searchable index will then be constructed using cryptographic functions.

4. Search Token Generation: When the user enters a keyword, a cryptographic search token will be generated using the secret key. The plaintext keyword will not be directly provided to the cloud server.

5. Encrypted Search: The cloud server will compare the search token with the protected index and identify the corresponding encrypted documents.

6. Result Retrieval: The matching encrypted files will be returned to the authorized client.

7. Decryption: The client will use the secret key to decrypt the retrieved files and obtain the original documents.

The performance of the proposed framework will be evaluated using encryption time, decryption time, search time, index construction time, storage overhead, and scalability for increasing numbers of encrypted documents.

V. Result

The proposed framework is expected to provide secure cloud-based file storage while retaining the ability to perform keyword searches over encrypted data. The system is expected to allow authorized users to retrieve relevant encrypted files without requiring the cloud server to access the plaintext contents of the stored documents.

The experimental evaluation is expected to demonstrate the relationship between the number of stored documents and the computational overhead associated with encryption, index construction, search, and decryption. The searchable index is expected to reduce the need for downloading and decrypting the entire cloud dataset for every search operation.

The proposed system is also expected to demonstrate the trade-off between search functionality and information leakage. Although the plaintext documents and search keywords will not be directly exposed to the cloud server, practical SSE systems may reveal limited information through search and access patterns. Recent research continues to address these limitations through forward privacy, backward privacy, verifiable search, efficient deletion, and improved index structures.

Overall, the proposed framework is expected to demonstrate that searchable symmetric encryption can provide a practical balance between confidential cloud storage and efficient encrypted-data retrieval.

VI. Flowchart

The complete workflow of the implemented system is shown in Fig. 1. The vault is first unlocked by deriving keys from the user password; every subsequent upload or search operation reuses those in-memory keys. The two operational paths — document upload and encrypted keyword search — share the same key hierarchy but exercise different modules of the system.

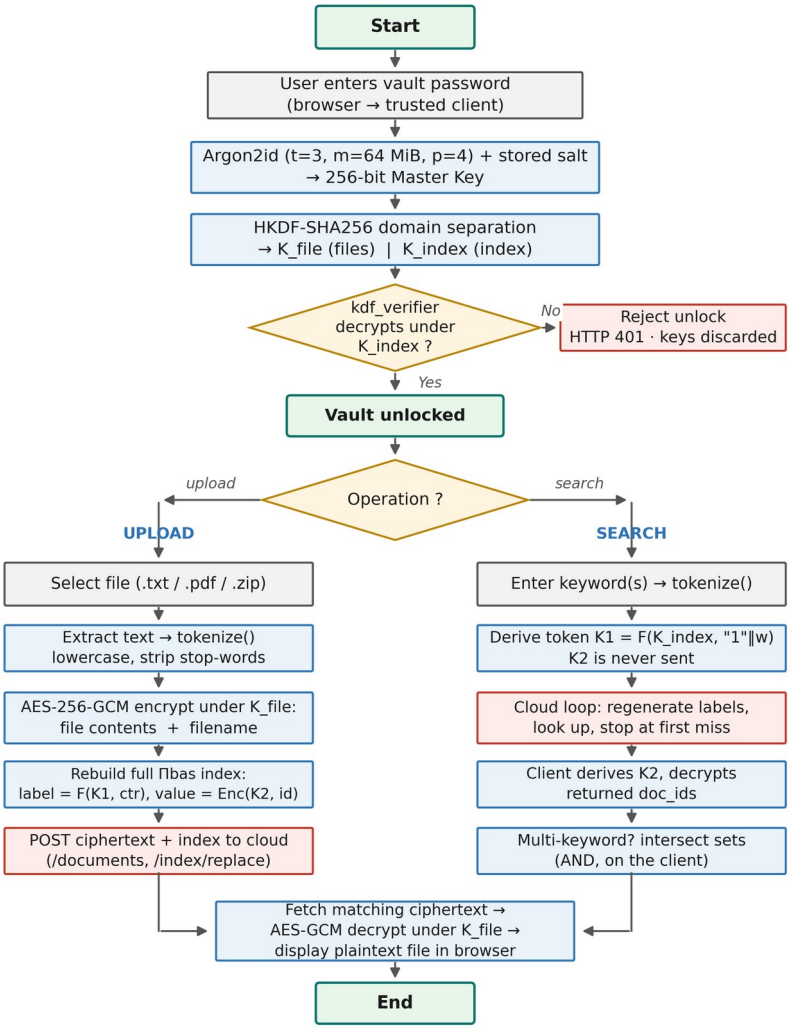


Figure 1: Complete workflow of the implemented system

On unlock, Argon2id combines the entered password with the salt stored on the cloud to produce a 256-bit master key, which HKDF-SHA256 expands into two independent sub-keys: K_file for document encryption and K_index for index and token derivation. An encrypted verifier value is then decrypted to confirm the password is correct; an incorrect password is rejected with HTTP 401 before any key reaches the upload or index logic. On upload, the file text is extracted and tokenized, the contents and filename are encrypted with AES-256-GCM, and the encrypted index is rebuilt and pushed to the cloud. On search, only the search token K1 is transmitted; the cloud regenerates labels and returns encrypted identifiers,

which the client decrypts locally using K2. For multi-keyword queries the resulting identifier sets are intersected on the client to implement the AND operation.

VII. Block Diagram — Working and Explanation

The implemented system runs as two genuinely separate operating-system processes on two different ports, so that the trust boundary is a real process boundary rather than an internal convention. The overall structure is shown in Fig. 2.

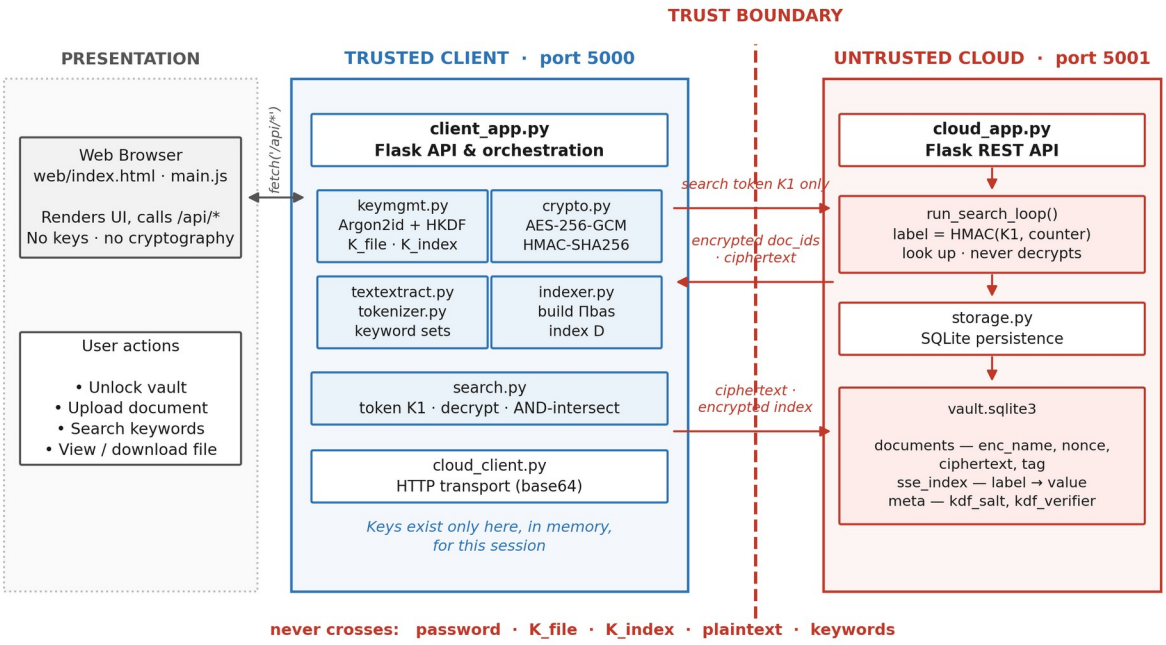


Figure 2: Block diagram of the implemented system

A. Major Components

Web browser (presentation layer): renders the interface and calls the client API over localhost. It performs no cryptography and never holds a key; all cryptographic work happens in the trusted client process.

client_app.py (trusted client): the Flask application that orchestrates unlock, upload, search, view, download and delete. It holds the derived keys in memory for the duration of the session.

keymgmt.py: derives the master key with Argon2id ($t = 3$, $m = 64$ MiB, $p = 4$) and expands it into K_file and K_index using HKDF-SHA256 with versioned domain-separation labels.

crypto.py: implements the three primitives used throughout — AES-256-GCM authenticated encryption, HMAC-SHA256 as the pseudorandom function F, and HKDF-SHA256 for key expansion.

textextract.py and tokenizer.py: extract readable text from .txt, .pdf and .zip inputs and normalise it into a deduplicated keyword set (lowercase, alphanumeric tokens, stop-words removed). The same tokenizer is used at index time and at search time, so the two can never drift apart.

indexer.py: constructs the flat Πbas encrypted index: for each keyword w it derives K1 and K2 from K_index, then emits label = F(K1, counter) and value = AES-GCM(K2, doc_id) for every matching document.

search.py: generates the search token K1 for a queried keyword, decrypts the identifiers returned by the cloud using K2, and intersects the result sets for multi-keyword queries.

cloud_client.py: the HTTP transport layer that base64-encodes ciphertext for transmission to the cloud process.

cloud_app.py (untrusted cloud): exposes the REST API and runs the search loop, regenerating labels with the supplied token and stopping at the first counter that is absent. It never decrypts anything.

storage.py and vault.sqlite3: persist three tables — documents (encrypted filename, nonce, ciphertext, tag), sse_index (label to encrypted value) and meta (KDF salt and verifier).

B. Data Flow

During upload, the client encrypts the file contents and filename under K_file, transmits the ciphertext to the cloud, rebuilds the complete encrypted index and replaces the stored index atomically. During search, the client transmits only the single-keyword token K1; the cloud regenerates the corresponding labels, returns the encrypted identifiers it finds, and the client decrypts them locally. Consequently the password, both sub-keys, the plaintext document contents and the plaintext keywords never cross the trust boundary — only ciphertext blobs and opaque HMAC tokens do.

VIII. Python Implementation — Working and Explanation

This section presents the core implementation. The listings below are abridged to the essential logic; supporting code such as validation, error handling and documentation comments has been omitted for brevity.

A. Cryptographic Primitives (client/crypto.py)

AES-256-GCM provides confidentiality and integrity in a single primitive. A fresh 96-bit nonce is generated for every encryption and no code path accepts a caller-supplied nonce, which structurally prevents nonce reuse. Decryption failures from a wrong key or tampered bytes are collapsed into one generic exception so that no padding-oracle-style side channel is created.

```
KEY_SIZE, NONCE_SIZE, TAG_SIZE = 32, 12, 16

def aes_gcm_encrypt(key, plaintext, associated_data=None):
    nonce = os.urandom(NONCE_SIZE)                 # fresh nonce every call
    ciphertext = AESGCM(key).encrypt(nonce, plaintext, associated_data)
    return nonce + ciphertext                      # nonce || ciphertext || tag

def aes_gcm_decrypt(key, blob, associated_data=None):
    nonce, ciphertext = blob[:NONCE_SIZE], blob[NONCE_SIZE:]
    try:
        return AESGCM(key).decrypt(nonce, ciphertext, associated_data)
    except InvalidTag as exc:
        raise DecryptionError('authentication failed') from exc

def prf(key, data):                                # F = HMAC-SHA256
    return hmac.new(key, data, hashlib.sha256).digest()
```

Listing 1: AES-256-GCM encryption, decryption and the PRF

B. Key Derivation (client/keymgmt.py)

The vault password is stretched into a 256-bit master key with Argon2id, which is memory-hard and therefore resistant to GPU-based brute forcing. HKDF-SHA256 then expands that master key into two cryptographically independent sub-keys using distinct info labels, so that exposure of one sub-key reveals nothing about the other.

```
def derive_master_key(password, salt, params=_DEFAULT_PARAMS):
    return hash_secret_raw(
        secret=password.encode('utf-8'), salt=salt,
        time_cost=params.time_cost,            # 3 iterations
        memory_cost=params.memory_cost_kib,    # 65536 KiB = 64 MiB
        parallelism=params.parallelism,        # 4 lanes
        hash_len=KEY_SIZE, type=Type.ID)       # Argon2id

def derive_subkeys(master_key):
    k_file  = hkdf_sha256(master_key, KEY_SIZE, salt=None, info=b'vault:k_file:v1')
    k_index = hkdf_sha256(master_key, KEY_SIZE, salt=None, info=b'vault:k_index:v1')
    return KeySet(k_file=k_file, k_index=k_index)
```

Listing 2: Argon2id key derivation and HKDF sub-key separation

C. Keyword Extraction (client/tokenizer.py)

A single tokenizer is used both when building the index and when processing a query. Because it is one function with two call sites, index-time and search-time normalisation cannot diverge, which is what guarantees that a search for “Encryption.” matches a document containing “encryption”.

```
_TOKEN_RE = re.compile(r'[A-Za-z0-9]+')
MIN_TOKEN_LENGTH = 2

def tokenize(text):
    keywords = set()
    for raw in _TOKEN_RE.findall(text.lower()):   # case-insensitive
        if len(raw) < MIN_TOKEN_LENGTH or raw in STOPWORDS:
            continue
        keywords.add(raw)                         # deduplicated per document
    return keywords
```

Listing 3: Deterministic keyword tokenizer

D. Encrypted Index Construction (client/indexer.py)

This is the core of the searchable encryption scheme. For every keyword, two keys are derived from K_index: K1 produces the lookup labels and K2 encrypts the document identifiers. Documents are processed in sorted order so that labels are reproducible, while the values use fresh nonces so that two rebuilds of an unchanged posting do not produce identical ciphertext — this prevents the cloud from comparing index snapshots to learn which entries were unchanged.

```
def keyword_label_key(k_index, keyword):    # K1 = F(K_index, '1' || w)
    return prf(k_index, b'1' + keyword.encode('utf-8'))

def keyword_value_key(k_index, keyword):    # K2 = F(K_index, '2' || w)
    return prf(k_index, b'2' + keyword.encode('utf-8'))

def build_index(k_index, documents):
    postings = {}
    for doc_id in sorted(documents.keys()):          # deterministic ordering
        for keyword in documents[doc_id]:
            postings.setdefault(keyword, []).append(doc_id)

    entries = []
    for keyword, doc_ids in postings.items():
        k1, k2 = keyword_label_key(k_index, keyword), keyword_value_key(k_index, keyword)
        for counter, doc_id in enumerate(doc_ids):
            label = prf(k1, counter_to_bytes(counter)).hex()   # F(K1, counter)
            value = aes_gcm_encrypt(k2, doc_id.encode('utf-8'))
            entries.append(IndexEntry(label=label, value=value))
    return entries
```

Listing 4: Construction of the flat ΠIbas encrypted index

E. Search Token and Result Decryption (client/search.py)

Only K1 is transmitted to the cloud. K2 is derived locally at the moment it is needed, because the cloud never decrypts anything and therefore has no use for it; withholding it narrows the key material that ever reaches the untrusted side.

```
def build_search_token(k_index, keyword):
    return keyword_label_key(k_index, keyword)     # K1 only -- K2 never sent

def decrypt_search_results(k_index, keyword, encrypted_values):
    k2 = keyword_value_key(k_index, keyword)       # derived locally
    return {aes_gcm_decrypt(k2, value).decode('utf-8') for value in encrypted_values}
```

Listing 5: Search-token generation and client-side result decryption

F. Cloud-Side Search Loop (cloud/cloud_app.py)

The cloud regenerates candidate labels from the supplied token and looks each one up, stopping at the first counter that is absent. It recognises which opaque rows exist but never learns the keyword, the identifiers or the document contents. The HMAC implementation is deliberately reimplemented here using the standard library so that the cloud package imports nothing from the client package.

```
def run_search_loop(conn, k1):
    results, counter = [], 0
    while True:
        label = _cloud_prf(k1, _counter_to_bytes(counter)).hex()
        value = storage.get_index_entry(conn, label)
        if value is None:              # first miss -> clean stop
            return results
        results.append(value)          # opaque ciphertext, never decrypted
        counter += 1
```

Listing 6: Cloud-side encrypted search loop

G. Upload Handling (client/client_app.py)

On upload the contents and the filename are encrypted separately under K_file before anything leaves the client. The encrypted index is then rebuilt over the whole corpus and replaced atomically, because adding one document can shift the posting counters of existing documents.

```
doc_id = os.urandom(16).hex()
file_blob = aes_gcm_encrypt(keys.k_file, raw_bytes)               # contents
name_blob = aes_gcm_encrypt(keys.k_file, file.filename.encode())  # filename
nonce, ciphertext, tag = _split_blob(file_blob)

cloud_client.push_document(doc_id, name_blob, nonce, ciphertext, tag)
_rebuild_and_push_index(keys)      # full rebuild + atomic /index/replace
```

Listing 7: Encrypting and uploading a document

IX. Output

The following outputs were captured from the completed system running as two separate processes — the trusted client on port 5000 and the untrusted cloud on port 5001 — using the eight-document sample corpus included with the project.

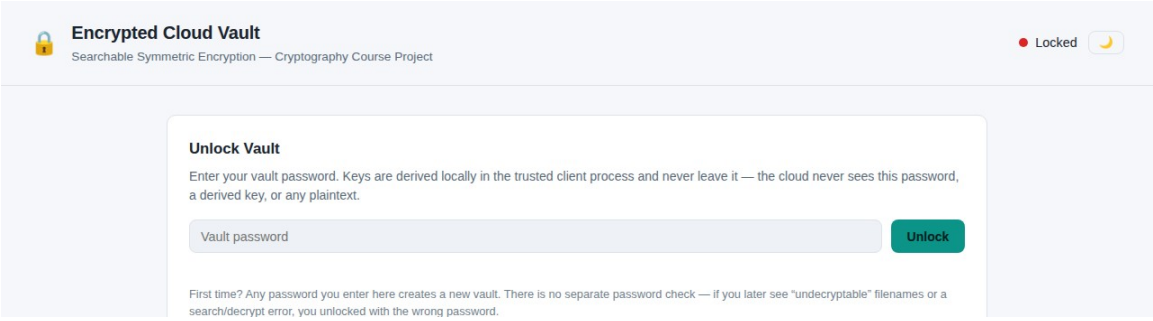


Figure 3: Initial locked state of the vault

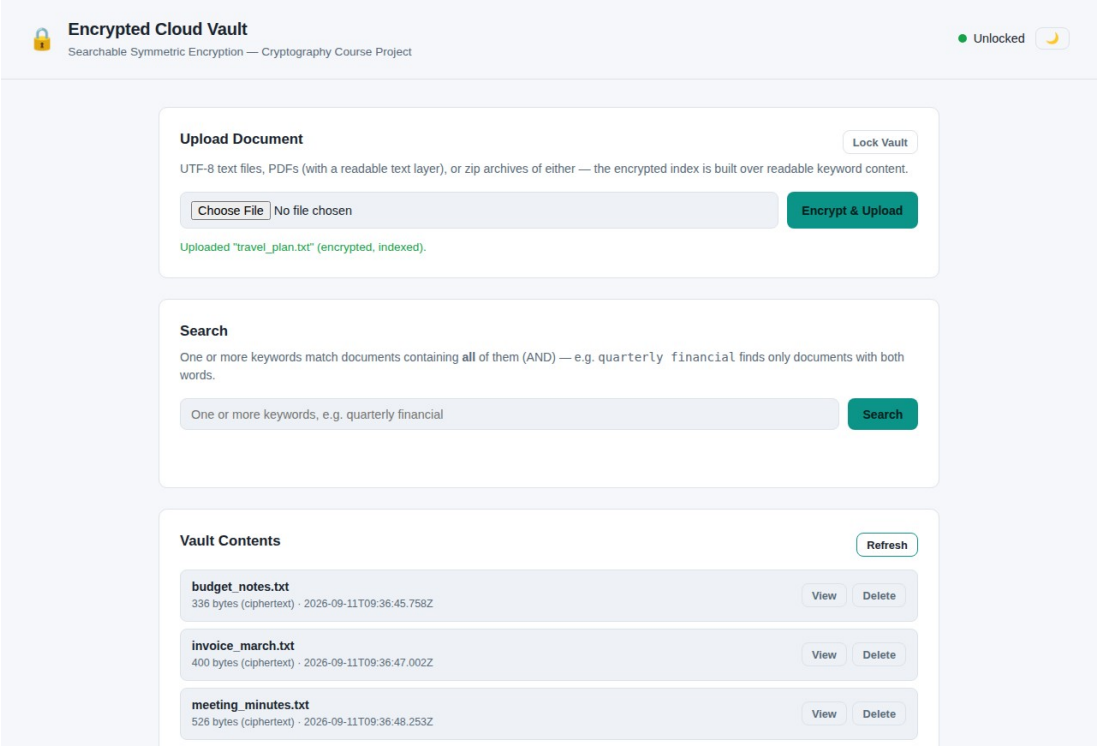


Figure 4: Vault contents after uploading the sample corpus (sizes shown are ciphertext sizes)

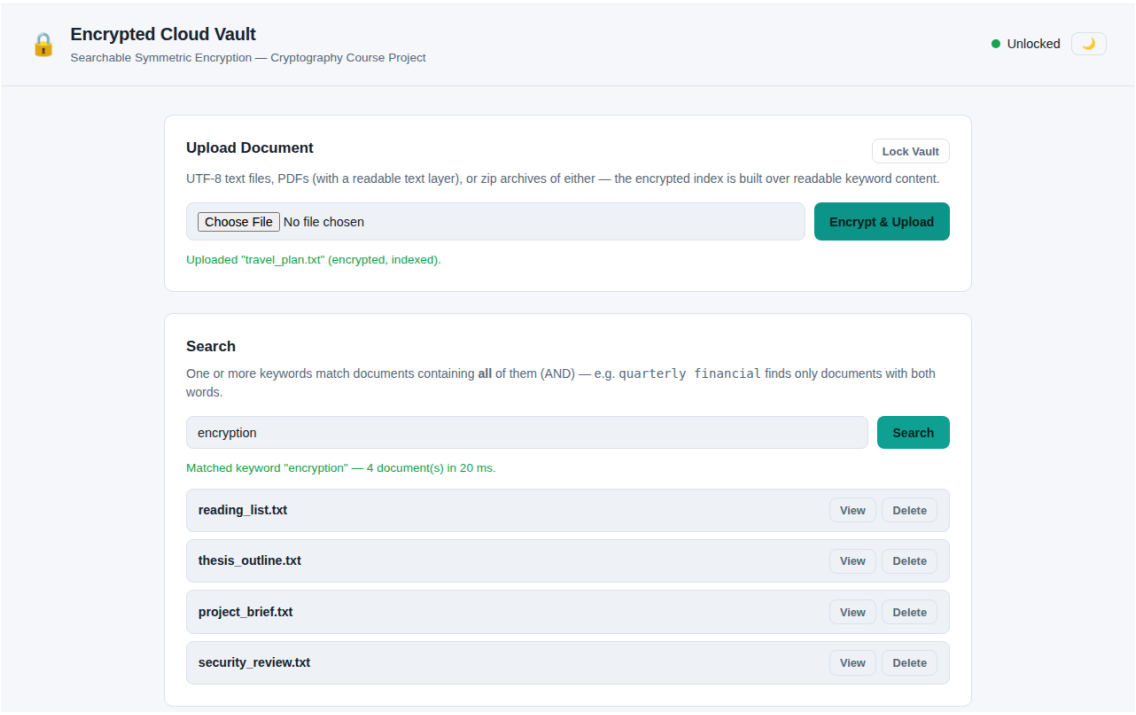


Figure 5: Single-keyword encrypted search returning four matching documents in 20 ms

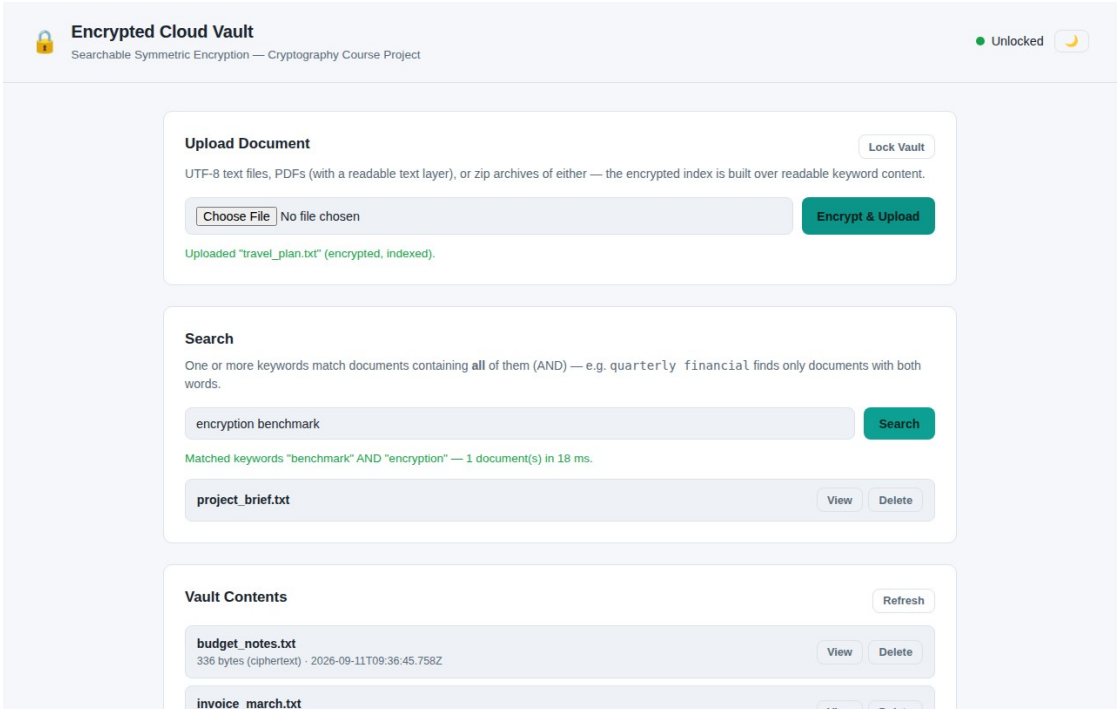


Figure 6: Multi-keyword (AND) search intersecting two keyword result sets on the client

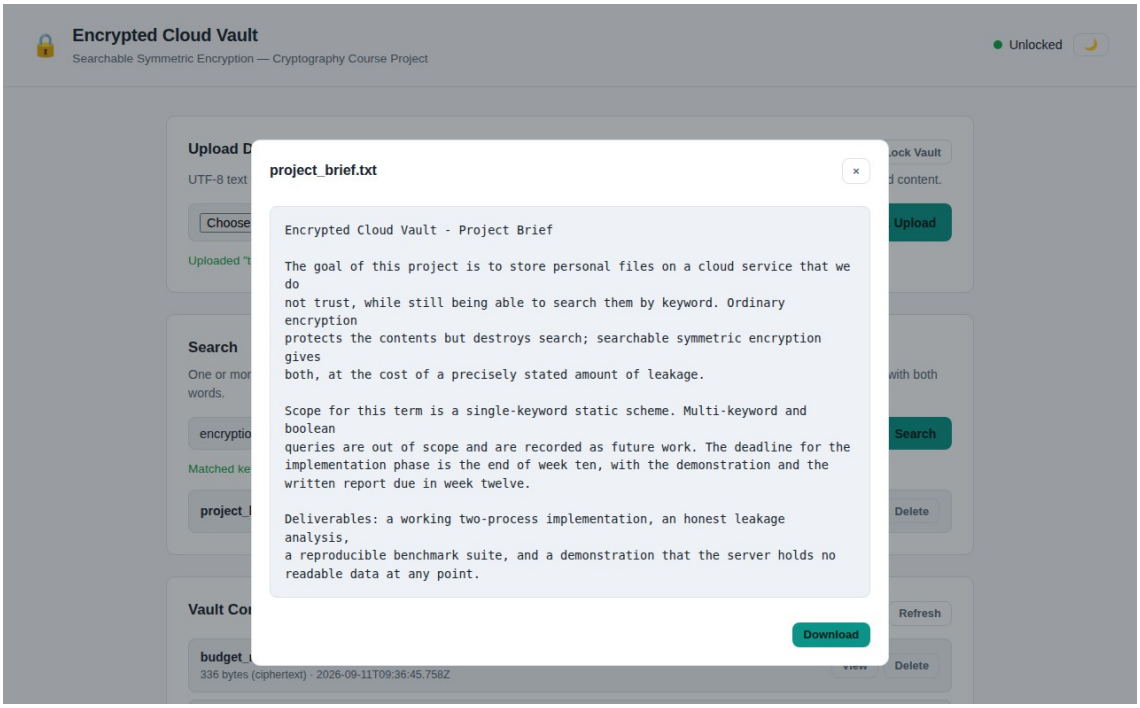


Figure 7: A matching document decrypted and displayed after retrieval

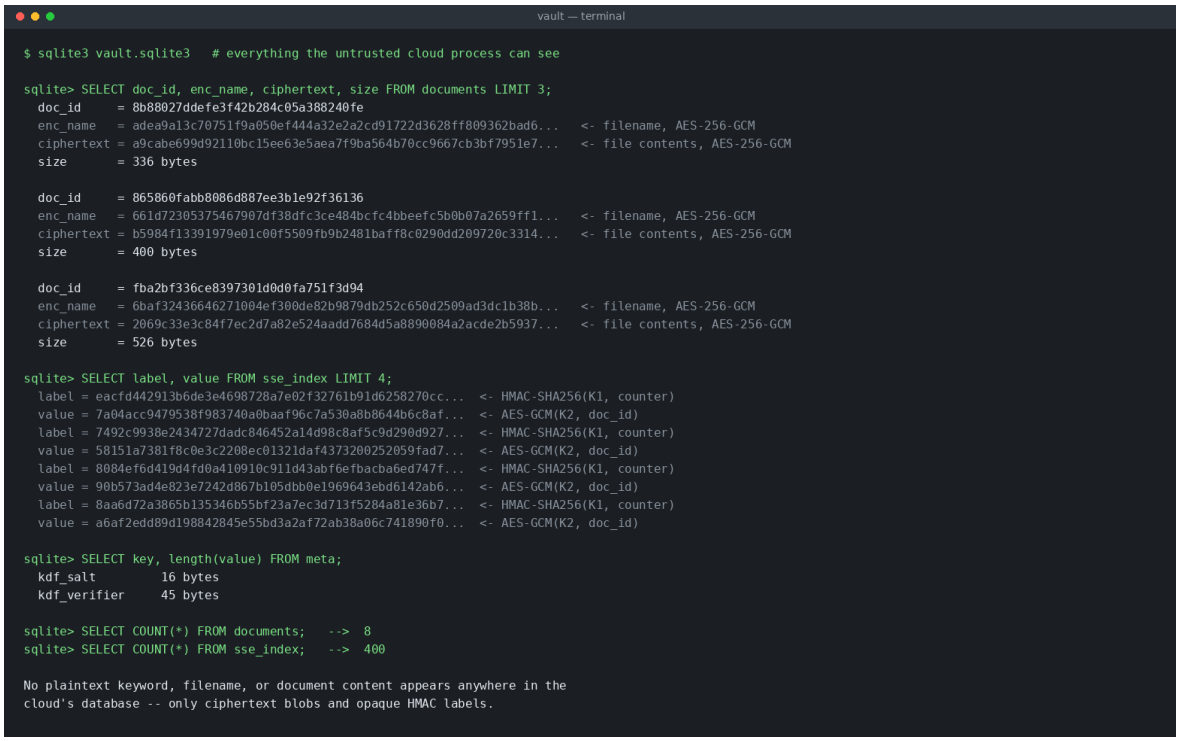


Figure 8: Contents of the cloud database — only ciphertext and opaque HMAC labels are stored

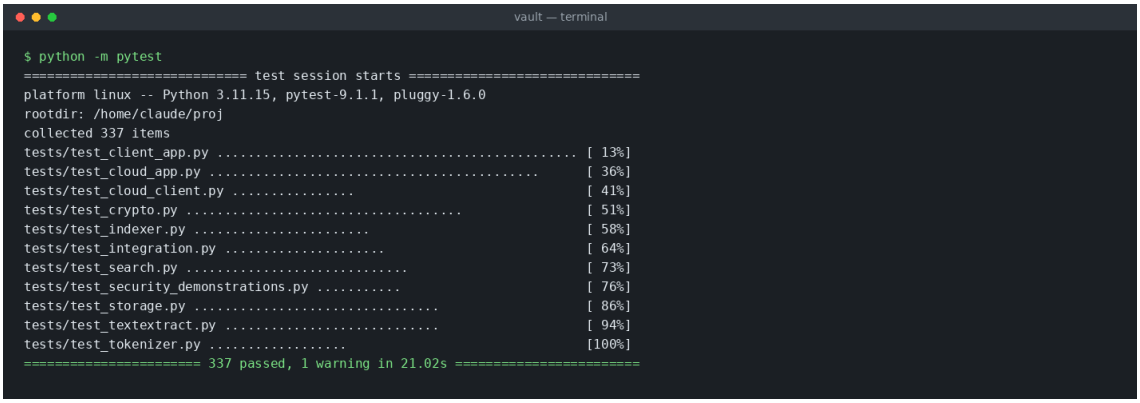


Figure 9: Final system output — complete test suite passing (337 tests)

Summary of Observed Results

The implemented system stored eight encrypted documents together with a 400-row encrypted index. A single-keyword search for “encryption” returned four matching documents in 20 ms, and a two-keyword conjunctive search for “encryption benchmark” correctly returned the single document containing both terms in 18 ms. Inspection of the cloud database confirmed that no plaintext filename, keyword or document content is present at any point; the cloud holds only AES-GCM ciphertext, opaque HMAC labels, the non-secret KDF salt and the password verifier. The complete automated test suite — covering the cryptographic primitives, the index, the search protocol, the trust boundary and the security demonstrations — passes with 337 tests.

X. References

[1] Z. Hu, J. Wang, Z. Chen, Z. Ge, M. Bian, L. Chen, and Y. Zhou, “SEAC: Dynamic Searchable Symmetric Encryption with Lightweight Update-Search Permission Control,” Cybersecurity, vol. 8, article 75, 2025.

[2] S. Xu, X. Chen, Y. Guo, Y. Yang, S. Wang, S.-M. Yiu, and X. Cheng, “Forward-Secure Multi-User and Verifiable Dynamic Searchable Encryption Scheme within a Zero-Trust Environment,” Future Generation Computer Systems, vol. 166, pp. 107701, 2025.

[3] L. Yang, Y. Yang, D. Niyato, Z. Li et al., “Dynamic Searchable Symmetric Encryption With Efficient and Complete Access Control for Multi-User Cloud Computing,” IEEE Transactions on Mobile Computing, 2025.

[4] “Forward and Backward Private Conjunctive Dynamic Searchable Symmetric Encryption With Refined Leakage Function and Low Communication,” IEEE Transactions on Information Forensics and Security, vol. 20, pp. 8224–8236, 2025.

[5] Z. Ren, P. He, R. Yu, L. Deng, Y. Tong, and S. Xu, “Dynamic Searchable Symmetric Encryption with Efficient Conjunctive Query and Non-Interactive Real Deletion,” Journal of Network and Computer Applications, vol. 245, article 104387, 2026.

[6] X. Zhang, H. Wang, J. Lu, T. Li et al., “A Bloom Filter-Based Dynamic Symmetric Searchable Encryption Scheme over Cloud Data,” Cybersecurity, vol. 9, article 47, 2026.

[7] X. Wang, J. Gong, K. Zhang, and H. Qian, “Decentralized Multi-Client Boolean Keyword Search for Encrypted Cloud Storage,” Computer Standards & Interfaces, vol. 97, article 104127, 2026.

[8] P. Wu, J. Shen, Z. Cao, and X. Dong, “MMKFB: Multi-Client and Multi-Keyword Searchable Symmetric Encryption with Forward and Backward Privacy,” Frontiers of Computer Science, vol. 19, no. 3, article 193804, 2025.

[9] A. Wu, D. Feng, M. Zhang, J. Chi, and Y. Zhang, “Efficient Verifiable Searchable Encryption with Search and Access Pattern Privacy,” Security and Safety, vol. 4, article 2024022, 2025.

[10] A. N. Khan, A. Naveed, A. Mehmood, D. Arora, A. ur R. Khan, and J. Ali, “BloomSec: Scalable and Privacy-Preserving Searchable Encryption for Cloud Environments,” PLOS ONE, vol. 20, no. 12, article e0336944, 2025.

[11] J. Xue, L. Zeng, W. Zhang, F. Li, X. Zhang, Y. Zhou, and L. Liu, “Incentive-Assisted Multi-Keyword Ranked Searchable Encryption with Hidden Attributes for Community Data Sharing,” Peer-to-Peer Networking and Applications, vol. 18, no. 3, article 153, 2025.

[12] H. M. Mohammed, M. H. Hashim, D. Kh. Altmemi, Z. A. Abduljabbar, Z. A. Hussien, V. O. Nyangaresi, A. A. Alyousif, and H. A. Neamah, “A Review: Searchable Symmetric Encryption Functionalities, Challenges and Solutions,” in Software Engineering: Emerging Trends and Practices in System Development, Lecture Notes in Networks and Systems, vol. 1562, Springer, Cham, pp. 47–65, 2026.